

AMGuard

Manual de Estudo — Volume 2 (Fase de Servir)

Dias 8, 9 e 10 — do notebook ao contêiner
FastAPI + Pydantic + Docker

Caio Bernardinelli — Técnico em IA (IFNMG) — Julho/2026
Continuação do Manual AMGuard (Vol. 1)

A fase de servir: do script ao serviço

Os Dias 1 a 7 transformaram o notebook em software — pacote Python, testes, portões de regressão. Mas até o final do Dia 7, o AMLGuard só existia como **scripts que você roda no seu Windows**. Se um recrutador quisesse experimentar, precisaria clonar o repo, instalar Python na versão certa, resolver dependências, treinar o modelo — fricção suficiente para fechar a aba em 30 segundos.

A fase de servir (Dias 8, 9, 10) resolve exatamente isso. No fim dela, qualquer pessoa em qualquer sistema operacional roda dois comandos e o AMLGuard sobe como serviço HTTP acessível pelo navegador — **sem instalar Python, sem instalar libs, sem treinar nada**. O modelo virou um produto que outros programas podem consultar.

ANALOGIA DO MUNDO REAL — abrir a padaria

Uma metáfora atravessa esta fase inteira: os Dias 1–7 foram desenvolver a receita do pão numa cozinha particular. Ninguém no bairro conseguia provar. A fase de servir é abrir a loja: uma porta, um balcão, uma placa. Cada dia acrescenta uma peça específica dessa transição.

Dia	Peça da padaria	O que entrega	Analogia central
8	Plaquinha ABERTO + etiqueta do pão	/health e /model-info	Fiscal e cliente confirmam antes de entrar
9	O balcão vendendo pão	/predict e /predict-batch com validação	Lanchonete de aeroporto com ficha padronizada
10	Franquia empacotada	Dockerfile e docker-compose	Contêiner de navio: cozinha completa portátil

Dia 8 — os endpoints administrativos (a plaquinha ABERTO)

O que — de script Python a serviço HTTP

Até o Dia 7, o AMLGuard era **um conjunto de scripts**. Você digitava `python -m src.models.predict`, ele rodava, retornava, terminava. Um usuário, uma máquina, uma execução por vez.

O Dia 8 transforma isso num **serviço** que fica no ar 24 horas por dia, escutando na porta 8000, pronto para responder pedidos vindos pela rede. Não expusemos ainda o `/predict` — isso é o Dia 9. Hoje foi só o alicerce: os dois endpoints administrativos que qualquer sistema profissional precisa antes de aceitar tráfego real.

ANALOGIA DO MUNDO REAL — script versus serviço

Script Python = uma receita anotada num caderno. Alguém pega o caderno, vai pra cozinha, prepara, come, acabou. Um usuário, uma cozinha, uma vez. Serviço HTTP = a mesma receita dentro de uma cozinha comercial ligada 24 horas com balcão de atendimento. Vários clientes ao mesmo tempo, sem tocar nas panelas, sem saber o que tem dentro. A cozinha (o processo Python) fica no ar; o balcão (o endpoint HTTP) atende.

Endpoint `/health` — a plaquinha ABERTO

Trabalho: retornar em ~1ms um JSON simples dizendo `"status: ok, service: amlguard-api, model_loaded: true/false, timestamp: agora"`. Zero side effects, zero cálculo pesado, zero risco.

Porquê: sistemas de orquestração (Docker, Kubernetes, load balancers de nuvem) batem no `/health` centenas de vezes por minuto para decidir três coisas: (1) essa instância está viva? (2) posso mandar tráfego real para ela? (3) devo reiniciar ela? Sem esse endpoint, a orquestração inteira que vem depois no plano (Dias 10, 11, 13) não teria em que se apoiar.

ANALOGIA DO MUNDO REAL — a plaquinha na porta

A padaria abre. Antes do fiscal da prefeitura entrar, do motoboy do delivery aceitar pedido, ou do cliente indeciso do outro lado da rua atravessar, todos olham a plaquinha ABERTO na porta. Ninguém pergunta ao padeiro se está aberto — a plaquinha diz. Barata, óbvia, indispensável.

Endpoint `/model-info` — a etiqueta do balcão

Trabalho: descrever o modelo carregado — nome, versão, threshold, hiperparâmetros, git commit de onde veio o treino, features esperadas. Tudo lido do `config.py` + `artifacts/model_metadata.json`.

Porquê: auditoria (o compliance oficial quer saber qual versão do modelo produziu um alerta) e clientes (o front-end quer mostrar "score calculado pelo modelo v0.1.0"). Se o modelo não está carregado, retorna 503 (Service Unavailable) com mensagem clara — sem enganar quem consulta.

ANALOGIA DO MUNDO REAL — etiqueta na vitrine

Você entra na padaria e há uma etiqueta no balcão: *"pão francês, fornada das 6h, farinha tipo 1"*. Você sabe o que vai receber sem perguntar. Se o padeiro ainda não fez a fornada do dia, a etiqueta diz "em preparo" — nem promete pão que não existe, nem esconde a informação.

Ferramenta central — FastAPI — o framework que faz Python virar HTTP

FastAPI é uma biblioteca Python que traduz suas funções em endpoints HTTP. Você escreve uma função que recebe um dicionário e devolve outro; ela converte isso automaticamente em "quando alguém fizer GET /health, chame essa função e devolva a resposta como JSON". Bônus poderoso: FastAPI gera automaticamente uma **documentação interativa** em /docs (interface Swagger UI) onde qualquer pessoa clica "Try it out" e testa os endpoints pelo navegador. Recrutador adora ver isso.

ANALOGIA DO MUNDO REAL — o tradutor simultâneo

Você fala português (Python nativo). O cliente fala HTTP (protocolo da web). O FastAPI é o tradutor simultâneo entre os dois, que também gera automaticamente um manual da conversa (o /docs) para quem chega novo.

Ideia crucial — lifespan e cache — por que o modelo carrega uma vez, não a cada request

O modelo XGBoost salvo tem ~5 MB. Carregar do disco leva ~100ms. Se cada request de /predict tivesse que abrir o joblib, seriam 100ms perdidos em cada chamada — inaceitável para produção. A solução: carregar UMA vez quando o servidor sobe (via o hook *lifespan*), guardar na memória como cache, e todos os requests reutilizam. O Dia 6 já tinha preparado essa infraestrutura (a variável `_MODEL_CACHE`); o Dia 8 só a conecta ao FastAPI.

ANALOGIA DO MUNDO REAL — acender o forno antes de abrir

A padaria não acende o forno cada vez que um cliente pede pão — acende às 5 da manhã, uma vez, e ele fica quente o dia todo. Os clientes se beneficiam da pré-aquecida. O *lifespan* é o "acende o forno na abertura", e o cache do modelo é o forno quente que atende todos.

LIÇÃO / ERRO REAL DO PROJETO — a pegadinha do 'from X import Y' (bug real do Dia 8)

Quando o teste rodou no Windows do Caio, dois casos falharam que passavam no Linux limpo. Motivo: quando o *main.py* faz *from src.models.predict import get_model*, ele COPIA a referência da função para o seu próprio namespace. Um *monkeypatch(predict.get_model)* nos testes não muda essa cópia. O fix: patchar também *src.api.main.get_model*. Regra geral do Python: **patch onde a função é USADA, não onde é DEFINIDA**. Bug clássico, vale gravar.

AUTOTESTE — responda antes de reler

- 1) Por que separar /health e /model-info do /predict em dois dias diferentes, e não implementar tudo junto?
- 2) Qual seria o impacto de recarregar o joblib a cada request /predict num serviço que recebe 1000 requests por minuto?
- 3) A pegadinha do monkeypatch: por que *from X import Y* quebra o patch mas *import X; X.Y()* não?

Dia 9 — o balcão vendendo pão (endpoints de predição)

O que — o primeiro contato com dados vindos de fora

Até o Dia 8, tudo era leitura de dentro do próprio sistema: `/health` lê variáveis internas, `/model-info` lê arquivos que o próprio time gera. O Dia 9 é a **fronteira**: pela primeira vez a API recebe dados vindos do mundo lá fora — JSON que qualquer cliente na internet pode enviar, com qualquer conteúdo, certo ou errado, honesto ou malicioso.

Foram entregues dois endpoints: **POST /predict** (uma transação por vez, para casos de uso em tempo real) e **POST /predict-batch** (até 1000 transações num único request, para processamento em lote noturno de compliance).

O trabalho principal NÃO é chamar `predict_transaction` — é validar o input

A chamada ao modelo XGBoost é uma linha só — `predict_transaction(dict)`, que existe desde o Dia 6. O trabalho de verdade do Dia 9 é **validar tudo o que chega antes de tocar no modelo**. Se alguém manda `amount_paid = -50`, ou um JSON sem o campo `payment_format`, ou o texto "muito" no lugar de um número, o serviço precisa parar na fronteira, retornar 422 (Unprocessable Entity) com mensagem clara, e não acordar o modelo.

ANALOGIA DO MUNDO REAL — a lanchonete de aeroporto

Compare duas lanchonetes. A lanchonete A tem um atendente que pega qualquer pedido gritado ("um X-tudo!"), passa pra cozinha, e a cozinha descobre no meio do preparo que o cliente queria sem cebola. Prato jogado fora, cliente esperando, cozinha bagunçada. A lanchonete B tem uma ficha padronizada: número de pães (obrigatório, 1–3), tipo de proteína (obrigatório, uma das 4 opções), extras (opcional). Cliente preenche a ficha antes de qualquer coisa. Ficha mal preenchida volta pro balcão IMEDIATAMENTE com "corrija isso". A cozinha nem toma conhecimento. Só fichas válidas chegam ao forno. O Dia 9 transformou o AMLGuard na lanchonete B.

Ferramenta central — Pydantic — a ficha padronizada

Pydantic é a biblioteca Python que descreve o formato exato que cada campo do JSON deve ter, e rejeita automaticamente qualquer JSON que não bata. Você define uma classe:

```
class TransactionRequest: payment_format: str; amount_paid: float (>= 0); sender_previous_tx_count: int (>= 0); is_business_hours: int (0 ou 1); same_account: int (0 ou 1)
```

e o FastAPI usa isso automaticamente. Toda tentativa de mandar JSON fora dessa forma volta com 422 **antes** de chegar à sua função. O bônus: o `/docs` mostra o formato exato esperado, e o cliente lê no navegador.

Ideia crucial — aliases — traduzindo entre nomes com espaço e Python

O modelo espera colunas com nome `"Payment Format"` e `"Amount Paid"` — nomes com espaço, herança do CSV IBM. Espaços não são identificadores Python válidos. A API pública, por outro lado, deve expor snake_case (`payment_format`, `amount_paid`) — convenção de REST. Pydantic tem o mecanismo de **aliases**: você declara a variável Python com o nome bonito e mapeia para o nome do CSV. A API aceita os dois formatos na entrada, e internamente traduz para o que o modelo precisa.

ANALOGIA DO MUNDO REAL — recepção bilíngue de hotel

A recepção do hotel aceita seu nome escrito em inglês (Caio) ou português (Caio) — dá no mesmo. Internamente, o sistema do hotel usa um código único (ID 42). O hóspede não precisa saber o código; a recepção traduz.

Ideia crucial — códigos de erro HTTP — os 3 status que importam hoje

200 OK — pedido válido, resposta com score. É o caminho feliz.

422 Unprocessable Entity — JSON estruturalmente OK mas com valores inválidos (campo faltando, tipo errado, número fora do intervalo). O Pydantic dispara isso automaticamente. Não é erro do sistema; é erro do cliente. A resposta vem com detalhes do que exatamente falhou (ex: "amount_paid: value must be >= 0").

503 Service Unavailable — o serviço não pode responder agora, por exemplo porque o modelo não está carregado. Também retornado se alguém tenta usar `/predict` antes do `lifespan` carregar o modelo.

ANALOGIA DO MUNDO REAL — as três respostas do balcão

200 = "aqui está seu pão". 422 = "seu pedido não faz sentido, corrija". 503 = "a cozinha está fechada agora". Três respostas distintas, três ações distintas do cliente. Confundir os três (por exemplo, retornar 200 com erro no corpo) confunde qualquer sistema automatizado tentando reagir.

Ideia crucial — limite de batch (max 1000) — por que não aceitar infinito

O `/predict-batch` tem um limite máximo de 1000 transações por request. Sem esse limite, alguém malicioso poderia mandar um request com 10 milhões de itens — o servidor tentaria processar tudo, memória estoura, CPU trava, o serviço fica indisponível para todos os outros usuários. Isso se chama **ataque de negação de serviço**. O limite é uma defesa: se o cliente precisar processar 100 mil transações, ele faz 100 requests de 1000, com pausa entre eles se necessário.

ANALOGIA DO MUNDO REAL — o limite de 12 itens no caixa expresso do supermercado

Existe por três motivos: previsibilidade do tempo de fila, proteção contra abuso, incentivo a comportamento cooperativo. Sem limite, um cliente com carrinho cheio trava a fila inteira. Mesma lógica no batch.

AUTOTESTE — responda antes de reler

- 1) Se o cliente manda `{"amount_paid": -50}`, o `/predict` deve retornar 400, 422 ou 500? Por quê?
- 2) Por que a validação do Dia 9 (Pydantic, na fronteira externa) precisa ser mais rigorosa que a validação de schema do Dia 2 (que valida um DataFrame já em memória)?
- 3) Se o limite de batch fosse 1 milhão em vez de 1000, quais seriam os três problemas concretos?

Dia 10 — a franquia empacotada (Docker)

O que — o problema que Docker resolve

Depois do Dia 9, o AMLGuard funciona — na sua máquina. Se um recrutador quiser rodar, precisa: (1) instalar Python 3.12 na versão certa, (2) instalar as libs, (3) resolver conflito com outras versões que ele já tenha, (4) ter o modelo treinado, (5) rodar o comando certo. Fricção suficiente para ninguém tentar.

Docker resolve isso empacotando tudo dentro de uma "caixa" (imagem) que contém: Python 3.12 específico, todas as libs específicas, o código, os arquivos de config. Qualquer pessoa em qualquer máquina (Linux, Mac, Windows, servidor de nuvem) executa `docker run amlguard` e recebe o serviço rodando idêntico — sem instalar nada além do próprio Docker.

ANALOGIA DO MUNDO REAL — do carrinho de hot-dog ao contêiner de navio

Antes de Docker: vender pão em outra cidade significava mudar. Levar forno, utensílios, contratar equipe local, adaptar receita ao gás daquela cidade, à farinha, à voltagem. Cada instalação era um projeto próprio. Depois: existe um contêiner-padrão de navio. Dentro dele você monta uma cozinha completa — forno, geladeira, insumos, receita colada na parede. Fecha o contêiner. Ele vai para qualquer porto. No destino, plugam numa tomada padrão e ele funciona igual. O Dockerfile é a planta dessa cozinha; a imagem é o contêiner selado; o `docker run` é o "pluga na tomada".

Dockerfile — a receita da imagem

Um arquivo de texto com instruções sequenciais: parta da imagem `python:3.12-slim`, instale as ferramentas de build, copie o `requirements.txt`, instale as libs Python, copie o código, defina o comando de startup (`uvicorn`). Cada instrução vira uma **camada** em cache — se você muda só o código mas não as dependências, o próximo build reaproveita 90% do trabalho anterior e leva 5 segundos em vez de 4 minutos.

ANALOGIA DO MUNDO REAL — a planta da cozinha

Uma planta arquitetônica não é a cozinha em si — é o desenho de como montá-la. Você pode dar a mesma planta pra 10 construtores em 10 cidades diferentes e os 10 vão montar cozinhas idênticas. O Dockerfile é essa planta; a imagem gerada é a cozinha construída.

Multi-stage build — por que a imagem final tem 671 MB e não 2 GB

O Dockerfile do AMLGuard tem duas fases: **builder** (imagem grande com toolchain de compilação, para instalar libs pesadas como XGBoost) e **runtime** (imagem enxuta, com só o `virtualenv` já pronto e o código). A fase builder fica no cache local (~2 GB) mas não sai da sua máquina; a fase runtime é o que vira a imagem final (671 MB). Quem baixar essa imagem só paga 671 MB, não 2 GB.

ANALOGIA DO MUNDO REAL — desmontar o andaime depois da obra

Durante a construção, você usa andaime, guindaste, cavalete, betoneira. Terminada a obra, você desmonta tudo e entrega o prédio. O andaime não fica no prédio. Multi-stage build é exatamente essa desmontagem: as ferramentas ficam no canteiro (builder), só o produto final é entregue (runtime).

.dockerignore — o gitignore do Docker

Sem `.dockerignore`, o `docker build` envia TUDO da pasta para o daemon Docker — incluindo o notebook de 1.8 MB, o `.git` de 1.3 MB, os assets, os testes, os arquivos temporários. O `.dockerignore` filtra: só o essencial (código `src/`, `requirements.txt`, `baseline_metrics.json` — 55 KB no total, 19 arquivos) chega ao build. Imagem final fica menor e mais segura (menos arquivos = menos superfície de ataque).

ANALOGIA DO MUNDO REAL — a mala do viajante

Você não coloca a caixa de vídeo cassete de 1998 na mala de mochilão. O `.dockerignore` é a triagem antes de fazer a mala: leva o que precisa, deixa o resto em casa.

docker-compose.yml — o comando único que orchestra

Em vez de digitar `docker run --rm -p 8000:8000 -v .....:ro amlguard:latest` com vários flags, o `docker-compose.yml` descreve isso uma vez, em YAML legível, e o comando vira só `docker compose up`. Além de porta e volume, define: reinício automático se cair, healthcheck integrado ao ciclo, nome amigável do container. É a lista de instruções que o gerente da franquia lê para abrir uma loja nova.

ANALOGIA DO MUNDO REAL — o manual de franquia

Quando a rede McDonald's abre uma nova franquia, o franqueado recebe um manual que descreve: mesa aqui, fritadeira ali, temperatura X do óleo, horário Y de troca de turno. Não improvisa. O `docker-compose.yml` é esse manual — descreve como montar a instância do serviço, e é executado por um comando único que qualquer franqueado consegue rodar.

Bind mount do modelo — por que o joblib não vai dentro da imagem

O `artifacts/model.joblib` não é copiado pelo Dockerfile. Em vez disso, no `docker run` ou no `compose`, ele é **bind-mounted**: o arquivo do seu disco é "emprestado" ao container em modo read-only. Motivo: separação entre **código** (versionado no Git, distribuído via imagem) e **modelo** (binário grande, gerado pelo treino, que muda com frequência independente do código). Trocar o modelo por uma versão retreinada não precisa de rebuild da imagem — só de reiniciar o container.

ANALOGIA DO MUNDO REAL — a farinha do dia

A cozinha da franquia vem com forno, panelas, receitas. Mas a farinha é comprada localmente todo dia e colocada no armário. Trocar de fornecedor de farinha não exige reconstruir a cozinha. Modelo = farinha do dia; código = cozinha.

HEALTHCHECK — o container que sabe se ele mesmo está saudável

O Dockerfile define uma instrução **HEALTHCHECK**: a cada 30 segundos, o próprio container faz um `GET /health` em si mesmo. Se responde 200, o container é considerado "healthy"; se não, "unhealthy". Orquestradores (Docker Compose, Kubernetes, ECS) leem esse status e reagem: reiniciar container morto, não mandar tráfego para instância unhealthy, avisar em dashboard.

ANALOGIA DO MUNDO REAL — o funcionário que verifica os próprios sinais vitais

É como o padeiro medindo a própria pressão a cada 30 minutos. Se ficar tonto, ele mesmo avisa o gerente e sai da linha. Ninguém precisa ficar vigiando de fora.

LIÇÃO / ERRO REAL DO PROJETO — descoberta importante — a diferença entre CONTENT SIZE e DISK USAGE

Quando o Caio rodou *docker images amlguard*, apareceu: CONTENT SIZE 671 MB, DISK USAGE 2.15 GB. Confusão possível: "minha imagem tem 2 GB??" NÃO. O **CONTENT SIZE** (671 MB) é o que sai da sua máquina se alguém baixar a imagem — é o número que importa para portfolio. O **DISK USAGE** (2.15 GB) é o Docker Desktop somando todas as camadas intermediárias em cache local, incluindo a fase builder. Não sai daqui.

AUTOTESTE — responda antes de reler

- 1) Por que o *.dockerignore* reduziu o contexto de ~4 MB para ~55 KB? O que ficou de fora?
- 2) Se você trocar uma linha de *src/api/main.py* e rodar *docker build* de novo, quais camadas vão do cache e qual vai ser refeita?
- 3) Por que o modelo é bind-mounted em vez de copiado para dentro da imagem? Cite dois motivos.
- 4) O que aconteceria se você removesse o HEALTHCHECK do Dockerfile e um container travasse silenciosamente em produção?

Fechamento da fase de servir

Ao final do Dia 10, o AMLGuard tem **três keywords de peso** que aparecem em toda vaga de MLOps/Data Engineering: **FastAPI + Pydantic + Docker**. Não como decoração no README, mas com prova de funcionamento: 44 testes verdes, imagem enxuta, container respondendo requests reais, /docs interativa acessível pelo navegador.

Antes da fase (fim do Dia 7)	Depois da fase (fim do Dia 10)
Scripts que só rodam na minha máquina	Serviço HTTP acessível na porta 8000
Recrutador precisa instalar Python + libs	Recrutador roda <i>docker run</i> e testa
Sem documentação de API	Swagger UI auto-gerada em /docs
"Funciona na minha máquina"	Funciona em qualquer máquina com Docker
44 testes verdes (predict, features, evaluate)	44 testes verdes + API real subindo em container

ANALOGIA DO MUNDO REAL — o insight sensorial do dia 10

Depois do *docker compose up*, quando o Caio abriu o navegador em <http://localhost:8000/docs> e clicou "Try it out" no /predict, ele viu um score voltar. Aquele score foi calculado por um XGBoost rodando dentro de uma caixa Linux isolada do Windows dele. Aquele momento é o que fica na memória sensorial e volta em entrevista: "eu já vi isso funcionar, sei o que é". Recrutador percebe.

Lições operacionais que valem para toda a carreira

Lição	Onde apareceu / por quê
Patch onde é USADO, não onde é DEFINIDO	Bug real do Dia 8. Ao fazer <i>from X import Y</i> , Python copia a referência. Monkeypatch em <i>X.Y</i> não muda a cópia. Se seu teste passa no Linux limpo mas quebra no Windows do colega, provavelmente é isso.
Validação de entrada vive na fronteira externa	Do Dia 9. Validação de DataFrame em memória (Dia 2) e validação de request JSON (Dia 9) são coisas diferentes. A segunda precisa ser mais rigorosa: o input pode ter vindo de qualquer lugar do planeta, com qualquer intenção.
Códigos HTTP não são detalhes cosméticos	200 vs 422 vs 503 são contratos com clientes automatizados. Sistemas de monitoramento, orquestradores e front-ends reagem diferente a cada um. Retornar 200 com erro no corpo confunde tudo.
Multi-stage build é obrigatório para imagens Python	Sem ele, uma imagem com XGBoost + sklearn passa fácil de 2 GB. Com ele, cabe em 500-700 MB — número aceitável para distribuir.
O modelo é dado, não código	Bind mount, não COPY. Trocar modelo (retreino, ajuste de threshold) não deveria exigir rebuild da imagem. Essa separação vale ouro em produção real.
HEALTHCHECK e /health não são a mesma coisa	O <i>/health</i> é o endpoint que responde. O HEALTHCHECK do Dockerfile é o container consultando o próprio /health e reportando status ao orquestrador. Os dois trabalham juntos — sem ambos, orquestração automática não funciona.
Docker CONTENT SIZE, não DISK USAGE	O primeiro é o que importa para portfolio (o que sai da máquina); o segundo é cache local. Não se assuste com o 2 GB do disk usage.

Ciclo de revisão sugerido para esta fase: 1 dia (autotestes), 1 semana (analogias + o bug do monkeypatch), 1 mês (o /docs no navegador ainda é o gancho sensorial mais forte — revisite abrindo e testando).